Lights Out Game - Arduino Code Breakdown

For Software Developers New to Arduino

Arduino Program Structure

Every Arduino sketch has two mandatory functions:

```cpp
void setup() {
  // Runs ONCE when the board powers on or resets
  // Initialize hardware, set pin modes, etc.
}

void loop() {
  // Runs REPEATEDLY forever (like a while(true) loop)
  // This is your main program logic
}
```

Hardware Components & Connections

What You Need:

- **LGT8F328P Board** (Arduino-compatible microcontroller)
- **WS2812 LED Strip** (8 NeoPixels) - Data pin connected to **D11**
- **8 Push Buttons** - Connected to pins **D2-D9**
- **Power Supply** - 5V for the board and LEDs

Button Wiring (Pull-up Configuration):

```
Button → Arduino Pin (D2-D9)
Button → GND (when pressed, connects pin to ground)
```

The `INPUT_PULLUP` mode means:

- **Not pressed**: Pin reads HIGH (5V via internal resistor)
- **Pressed**: Pin reads LOW (0V, connected to ground)

Key Arduino Concepts

1. Pin Modes

```cpp
pinMode(pin, INPUT_PULLUP);  // Configure pin as input with internal pull-up resistor
```

2. Digital Read

```cpp
digitalRead(pin);  // Returns HIGH or LOW
```

3. Serial Communication

```cpp
Serial.begin(9600);      // Initialize serial at 9600 baud rate
Serial.println("text");  // Send text to Serial Monitor for debugging
```

4. Delays

```cpp
delay(100);  // Pause for 100 milliseconds (blocks execution)
```

Code Structure Breakdown

1. Library & Hardware Setup

```cpp
#include <FastLED.h>

#define LED_PIN 11
#define NUM_LEDS 8
#define BUTTON_START 2

CRGB leds[NUM_LEDS];  // Array to hold color data for each LED
```

FastLED Library: Handles WS2812 LED communication. The library takes care of precise timing requirements that would be difficult to implement manually.

---

2. Global State Variables

```cpp
bool ledStates[NUM_LEDS];      // Game state: which LEDs should be on
bool buttonPressed[NUM_LEDS];  // Track which buttons are currently held
```

Why global? In Arduino, state must persist across `loop()` iterations. Unlike desktop apps with event loops, Arduino's `loop()` runs continuously, so we store state globally.

---

3. setup() - Initialization

```cpp
void setup() {
  Serial.begin(9600);  // Enable Serial Monitor for debugging

  // Initialize LED strip
  FastLED.addLeds<WS2812, LED_PIN, GRB>(leds, NUM_LEDS);
  FastLED.setBrightness(50);

  // Configure button pins
  for (int i = 0; i < NUM_LEDS; i++) {
    pinMode(BUTTON_START + i, INPUT_PULLUP);
  }

  randomSeed(analogRead(A0));  // Seed random generator with analog noise
  newGame();  // Start first game
}
```

Key Points:

- `FastLED.addLeds<WS2812, LED_PIN, GRB>()` - Tells library what type of LEDs, which pin, and color order
- `INPUT_PULLUP` - Enables internal resistor (no external resistor needed)
- `randomSeed()` - Uses electrical noise on unconnected analog pin for randomness

---

4. loop() - Main Game Logic

```cpp
```

```cpp
void loop() {
  // Poll each button every loop iteration (~100-1000 times per second)
  for (int i = 0; i < NUM_LEDS; i++) {
    bool currentState = (digitalRead(BUTTON_START + i) == LOW);

    // Edge detection: detect press (LOW) after release (HIGH)
    if (currentState && !buttonPressed[i]) {
      buttonPressed[i] = true;
      handleButtonPress(i);
      updateDisplay();
      delay(50);  // Simple debounce
    }
    else if (!currentState && buttonPressed[i]) {
      buttonPressed[i] = false;  // Button released
    }
  }

  if (checkWin()) {
    celebrateWin();
    delay(2000);
    newGame();
  }

  delay(10);  // Stabilize readings
}
```

Important Concepts:

- **Polling**: Unlike event-driven GUI apps, Arduino continuously checks button states
- **Edge Detection**: We detect the *transition* from unpressed to pressed (not just "is pressed")
- **Debouncing**: Physical buttons "bounce" when pressed (multiple signals). The 50ms delay prevents multiple triggers

---

5. Game Logic - handleButtonPress()

```cpp
```

```cpp
void handleButtonPress(int button) {
    // Toggle pressed LED
    ledStates[button] = !ledStates[button];

    // Toggle neighbors
    if (button > 0) {
        ledStates[button - 1] = !ledStates[button - 1];
    }
    if (button < NUM_LEDS - 1) {
        ledStates[button + 1] = !ledStates[button + 1];
    }
}
```

Lights Out Rule: Pressing a button toggles itself and adjacent LEDs.

---

6. Display Update

```cpp
void updateDisplay() {
    for (int i = 0; i < NUM_LEDS; i++) {
        leds[i] = ledStates[i] ? colorOn : colorOff;
    }
    FastLED.show();  // Send data to LED strip
}
```

Important: LEDs don't update until `FastLED.show()` is called. This allows you to prepare all colors in memory before sending them out in one batch.

---

7. Puzzle Generation

```cpp
```

```cpp
void newGame() {
  if (useCustomPattern) {
    // Copy user-defined pattern
    for (int i = 0; i < NUM_LEDS; i++) {
      ledStates[i] = customPattern[i];
    }
  } else {
    // Generate solvable puzzle by simulating random button presses
    int numPresses = random(5, 15);
    for (int i = 0; i < numPresses; i++) {
      int button = random(NUM_LEDS);
      // Apply toggle logic (same as handleButtonPress)
      ledStates[button] = !ledStates[button];
      if (button > 0) ledStates[button - 1] = !ledStates[button - 1];
      if (button < NUM_LEDS - 1) ledStates[button + 1] = !ledStates[button + 1];
    }
  }
  updateDisplay();
}
```

Why simulate button presses? This guarantees the puzzle is solvable. Any pattern created by button presses can be solved by reversing those presses.

Important Parts for Building This System

1. Pin Configuration (Lines ~20-22)

```cpp
#define LED_PIN 11
#define NUM_LEDS 8
#define BUTTON_START 2
```

Change these if your wiring is different.

---

2. LED Type Configuration (Line ~44)

```cpp
FastLED.addLeds<WS2812, LED_PIN, GRB>(leds, NUM_LEDS);
```

- `WS2812` - LED chip type (also works for WS2812B)

- `GRB` - Color order (some strips use `RGB` instead)

If colors are wrong, change `GRB` to `RGB` or `BGR`.

---

3. User Configuration (Lines ~27-35)

```cpp
bool useCustomPattern = false;  // Set to true for custom pattern
bool customPattern[NUM_LEDS] = {1, 1, 0, 0, 1, 1, 0, 0};
```

Easy customization without understanding the entire code.

---

4. Brightness (Line ~45)

```cpp
FastLED.setBrightness(50);  // 0-255, adjust for comfort
```

---

5. Debounce Delay (Line ~72)

```cpp
delay(50);  // Increase if buttons trigger multiple times
```

Common Arduino Pitfalls for Software Developers

1. **No Dynamic Memory Allocation**: Avoid `malloc()` or `new` on Arduino. Use fixed-size arrays.
2. **Limited RAM**: Arduino Uno/LGT8F328P has only **2KB RAM**. Strings and large arrays consume memory quickly.
3. **Blocking Code**: `delay()` stops everything. For complex apps, use `millis()` for non-blocking timing:

```cpp
```

```
unsigned long currentTime = millis();
if (currentTime - lastTime > 1000) {
  // Do something every 1 second
  lastTime = currentTime;
}
```

4. **No Operating System**: No threads, no async/await. Everything runs sequentially in `loop()`.
5. **Serial Monitor is Your Debugger**: Use `Serial.println()` liberally since there's no traditional debugger.

Testing & Debugging Tips

1. **Open Serial Monitor** (Tools → Serial Monitor, 9600 baud)
2. **Test buttons individually** with the button test sketch first
3. **Check LED colors** - if wrong, change color order in FastLED setup
4. **Measure voltage** - WS2812 LEDs need 5V, will flicker or not work at 3.3V
5. **Power considerations** - 8 LEDs at full brightness can draw ~480mA. USB can provide this, but for longer strips use external power.

Further Modifications

- **Change colors**: Modify `colorOn` and `colorOff` (e.g., `CRGB::Blue`)
- **Add difficulty levels**: Adjust `random(5, 15)` for more/fewer starting LEDs
- **Add sounds**: Use `tone()` function with a buzzer
- **Add timer**: Track solve time with `millis()`
- **Extend to 2D grid**: Use an 8x8 LED matrix for traditional Lights Out

---

Resources

- **FastLED Documentation**: http://fastled.io
- **Arduino Language Reference**: https://www.arduino.cc/reference/en/
- **WS2812 Datasheet**: For understanding LED protocol timing